Vision 기본 모델 Day 5

세그멘테이션 · 포즈 · ONNX · 성능 측정 · 위험구역 침입 감지

HYUNDAI AI Insight Campus · 온디바이스 AI · Vision 기본 모델 5/5 (마지막 날)

2026. 11. 3 (화) 09:00–18:00 · 8 교시 · Google Colab T4 GPU (측정은 CPU)

세그멘테이션 | mIoU | 포즈 17 관절 | ONNX Runtime | latency · FPS | 위험구역 침입 감지

" 어디에 " → " 어떤 모양 · 어떤 자세 " → " 얼마나 빠르게 "

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 박스의 한계

Penn-Fudan 마스크 PNG 로 박스 안 사람 비율과 박스 비율 (h/w) 을 재 보고 , 박스만으로 부족한 상황을 설명한다

2 세그멘테이션 · mIoU

semantic vs instance, Encoder-Decoder · Transposed Conv 를 이해하고 YOLOv8n-seg 마스크를 mIoU 로 평가한다

3 포즈 · 쓰러짐 규칙

COCO 17 관절을 읽고 관절 각도 · 몸통 기울기로 쓰러짐 규칙 (기울기 > 60°) 을 만든다

4 ONNX export

MobileNetV3 · YOLOv8n 을 ONNX 로 내보내고 , ONNX Runtime 결과가 PyTorch 와 같은지 확인한다

5 성능 측정 · baseline

warm-up · 평균 · p50 · p95 · FPS · 스레드 규칙으로 측정해 baseline.csv 를 만든다

6 위험구역 침입 감지

ONNX YOLOv8n + 발 위치 + ROI 다각형 + 3 프레임 debounce 로 경보 영상과 CSV 로그를 만든다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 오전 세그 → 포즈 , 오후 태스크 정리 → ONNX → 측정 → 침입 감지 → 발표

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 4 복습 퀴즈 · 박스로는 부족한 상황	퀴즈 + 강의	D5-1 박스 vs 마스크 vs 관절
2	10:00–10:50	세그멘테이션 — Encoder-Decoder · mIoU	강의 + 실습	D5-2 Encoder-Decoder 와 mIoU
3	11:00–11:50	포즈 추정 — 관절 · 쓰러짐 규칙	강의 + 실습	D5-3 관절과 쓰러짐 규칙
4	13:00–13:50	비전 태스크 정리 — 무엇을 고를까	강의 + 실습	D5-4 태스크별 출력 텐서
5	14:00–14:50	ONNX export · ONNX Runtime	강의 + 실습	D5-5 ONNX 그래프
6	15:00–15:50	성능 측정법 — latency · FPS · 스레드	강의 + 실습	D5-6 latency 분포와 스레드
7	16:00–16:50	종합 미니 프로젝트 — 위험구역 침입 감지	실습	D5-7 ROI 다각형과 침입 판정
8	17:00–17:50	발표 · 최종 퀴즈 30 문항 · 마무리	발표 + 퀴즈	D5-8 5 일 지도

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 검증 (노트북) → 점검 (워크시트) — 모두 브라우저에서 바로 열립니다

기본 주소 (Day 5)

https://imguru-mooc.github.io/AI_CAMPUS/VB-day5/

animation.html	애니메이션 8 개 (D5-1 ~ D5-8) — #d5-3 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기
VB_Day5_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 종합 미니 프로젝트 (Colab · T4 GPU)
VB_Day5.pdf	오늘 슬라이드 PDF
VB-day4/data/people.mp4	Day 4 보행자 영상 그대로 — 768×576 · 10 FPS · 795 프레임 · 7 교시
Day 4 복습 퀴즈	09:00 시작 — 강사가 Google Forms 링크 공유 (20 문항 · 15 분)
최종 퀴즈 30 문항	8 교시 — Day 1~5 전체 필기 퀴즈 (Google Forms)

노트북은 " 파일 → Drive 에 사본 저장 " 후 작업 · 런타임 → T4 GPU · 성능 측정 (6~7 교시) 은 CPU 기준 · 마지막 날 : baseline.csv 제출

오늘의 큰 그림 — 박스에서 마스크 · 관절 , 그리고 배포

" 어디에 " → " 어떤 모양 · 어떤 자세 " → " 얼마나 빠르게 " · 세그 → 포즈 → ONNX → 측정 → 침입 감지



오늘의 데이터 · 모델

- Penn-Fudan — 170 장 · 423 명 · 마스크 PNG (PedMasks/*_mask.png, 0 = 배경 , 1, 2, … = 사람 번호)
- people.mp4 (Day 4 영상) — 768×576 · 10 FPS · 795 프레임
- ultralytics YOLOv8n 계열 (COCO 사전학습) — 탐지 · 세그 · 포즈
- 참고 (Colab 선택): DeepLabV3-MobileNetV3-Large (semantic) · Keypoint R-CNN (top-down)
- 학습 · 추론은 T4 GPU, 성능 측정은 CPU (Colab 2 코어) — 온디바이스 근사

같은 YOLOv8n 몸통에 head 만 다르다 — 640 입력 , 8,400 칸

모델	과제	파라미터	원시 출력 (640 입력)
yolov8n.pt	탐지	3,157,200	(1, 84, 8400) = 박스 4 + 클래스 80
yolov8n-seg.pt	인스턴스 세그	3,409,968	(1, 116, 8400) = 4 + 80 + 마스크 계수 32 · prototype (1, 32, 160, 160)
yolov8n-pose.pt	포즈	3,295,470	(1, 56, 8400) = 4 + 사람 점수 1 + 17 관절 × 3

1 Day 4 복습 · 박스로는 부족한 상황

09:00–09:50

학습 목표

- 09:00–09:15 Day 4 복습 퀴즈 (20 문항)
- 박스 안 상당 부분은 배경
- 위험구역 경계 침범 = 몸이 닿았나 · 발이 들어갔나
- 쓰러짐은 박스 비율 (h/w) 로는 애매

노트북 실습

1. 문제 1-1 마스크 PNG 읽기
2. 문제 1-2 박스 안 사람 비율
3. 문제 1-3 박스 비율의 한계

▶ 애니메이션 D5-1 박스 vs 마스크 vs 관절

핵심 개념

1-1 마스크

```
PedMasks/FudanPed00001_mask.png
np.unique(mask) → [0, 1, 2]
0 = 배경 · 1, 2 = 사람 번호 (인스턴스) · mask == 1 → 첫 번째 사람
```

1-2 사람 비율

```
fill = 박스 안 사람 픽셀 수 / 박스 넓이
Penn-Fudan 평균 0.5036 (실행 예) → 나머지는 배경
```

1-3 박스 비율

```
서 있는 사람 h/w 평균 2.77 (실행 예) · 누우면 < 1
허리 숙임 · 카메라 각도에도 바뀐다 → 쓰러짐 판정엔 애매
```

출력 비교

```
분류  라벨 1개
탐지  (클래스, 박스, 점수)
세그  픽셀마다 클래스 (semantic) / 물체마다 마스크 (instance)
포즈  사람마다 관절 17개 (x, y, conf)
```

체크포인트

박스는 "어디에" 까지만 알려 준다. 모양이 필요하면 마스크 (세그멘테이션), 자세가 필요하면 관절 (포즈) 이 필요하다.

2 세그멘테이션 — Encoder-Decoder · mIoU

10:00–10:50

학습 목표

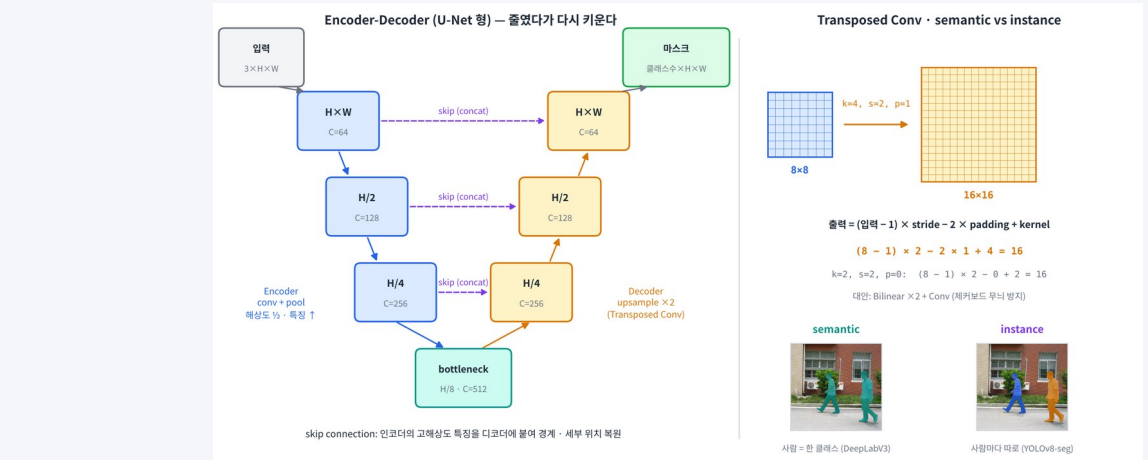
- semantic = 픽셀마다 클래스 (DeepLabV3)
- instance = 물체마다 마스크 (Mask R-CNN · YOLOv8-seg)
- Encoder 줄이고 → Decoder 키운다 · skip (U-Net)
- mIoU = 클래스별 IoU 의 평균

노트북 실습

1. 문제 2-1 tconv_out · ConvTranspose2d
2. 문제 2-2 miou
3. 문제 2-3 YOLOv8n-seg 마스크
4. 문제 2-4 Penn-Fudan mIoU

▶ 애니메이션 D5-2 Encoder-Decoder 와 mIoU

핵심 개념





체크포인트

실행 예 (YOLOv8n-seg · 170 장): 배경 IoU 0.9425 · 사람 IoU 0.7682 · mIoU 0.8553 — 마스크 = 계수 32 × prototype 32 장 → sigmoid → 박스 안 .

3 포즈 추정 — 관절 · 쓰러짐 규칙

11:00–11:50

학습 목표

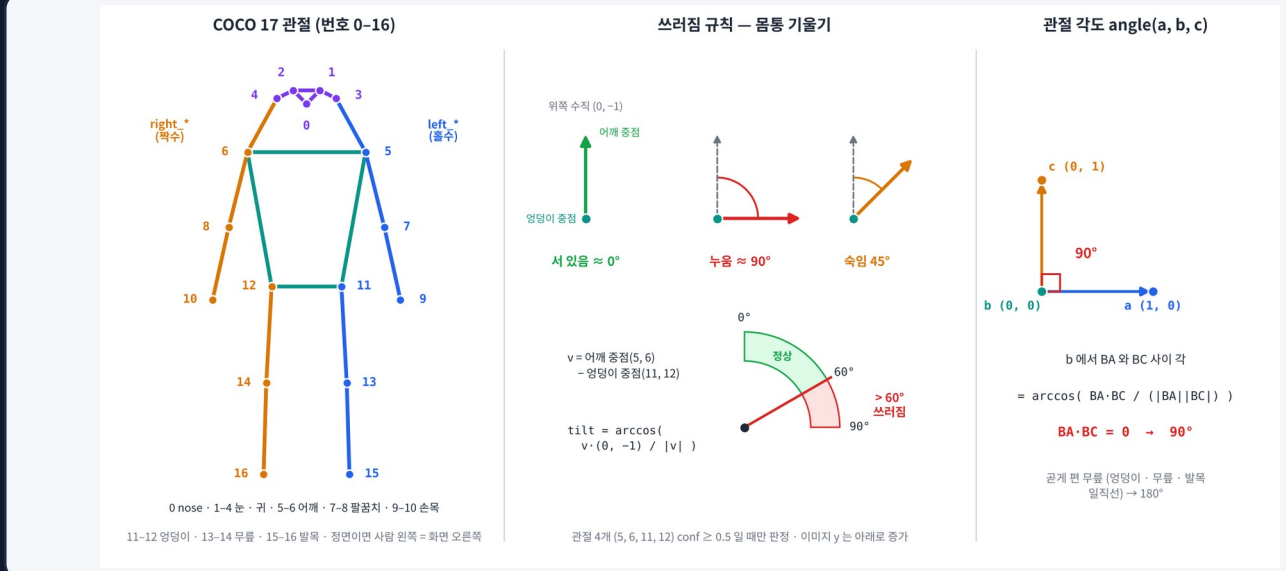
- COCO 17 관절 (x, y, conf)
- heatmap vs 직접 회귀 (YOLOv8-pose)
- top-down (사람 수만큼 느려짐) vs bottom-up / 1-stage
- 몸통 기울기 > 60° → 쓰러짐

노트북 실습

1. 문제 3-1 keypoints 읽기
2. 문제 3-2 angle
3. 문제 3-3 body_tilt · is_fallen

▶ 애니메이션 D5-3 관절과 쓰러짐 규칙

핵심 개념



3-3 (실행 예)

```
Penn-Fudan 사진 90° 회전 → ' 누운 사람 ' 흉내
원본 기울기 3.3° · 회전 84.2°
```

체크포인트

top-down (Keypoint R-CNN · HRNet) 은 정확하지만 사람 수만큼 느려지고 , YOLOv8-pose 는 사람 수에 상관없이 한 번에 예측한다 .

4 비전 태스크 정리 — 무엇을 고를까

13:00–13:50

학습 목표

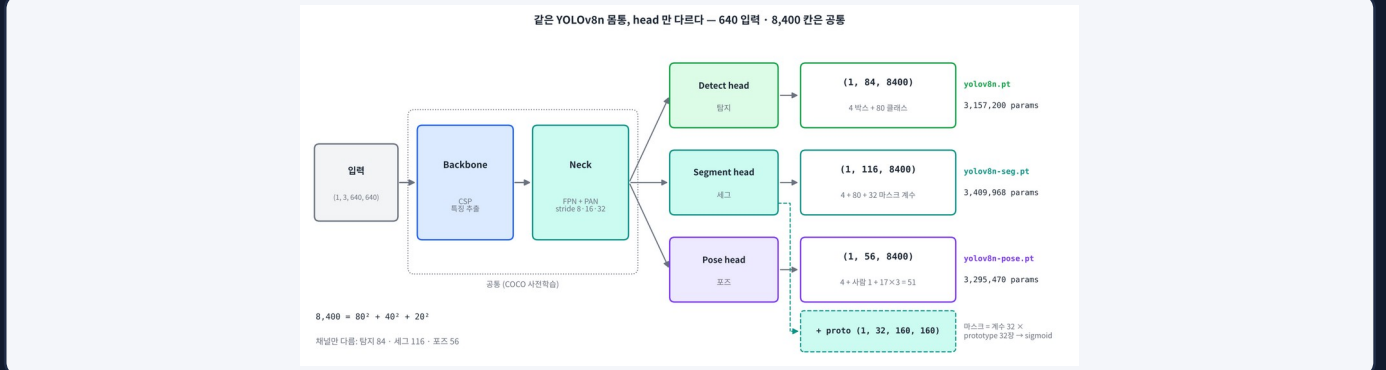
- 같은 YOLOv8n 몸통 + head 만 다름
- 출력 채널 : 탐지 84 · 세그 116 · 포즈 56
- 8,400 칸은 같다
- 필요한 출력 → 가장 단순한 태스크 → 속도

노트북 실습

1. 문제 4-1 태스크별 출력 텐서
2. 문제 4-2 시나리오 매칭

▶ 애니메이션 D5-4 태스크별 출력 텐서

핵심 개념



4-2 시나리오	태스크
제품 사진 한 장의 양품 / 불량 판정	분류
작업장 출입 인원 세기	탐지
지게차와 사람 사이 거리 경고	탐지
바닥 기름 유출 영역의 넓이 재기	세그멘테이션
작업자 쓰러짐 감지	포즈
크레인 아래 위험구역에 발을 들였나	탐지 + ROI (발 위치)

체크포인트

먼저 필요한 출력이 " 있나 / 어디 / 어떤 모양 / 어떤 자세 " 중 무엇인지 정하고 , 그중 가장 단순한 태스크를 고른 뒤 속도를 본다 .

5 ONNX export · ONNX Runtime

14:00–14:50

학습 목표

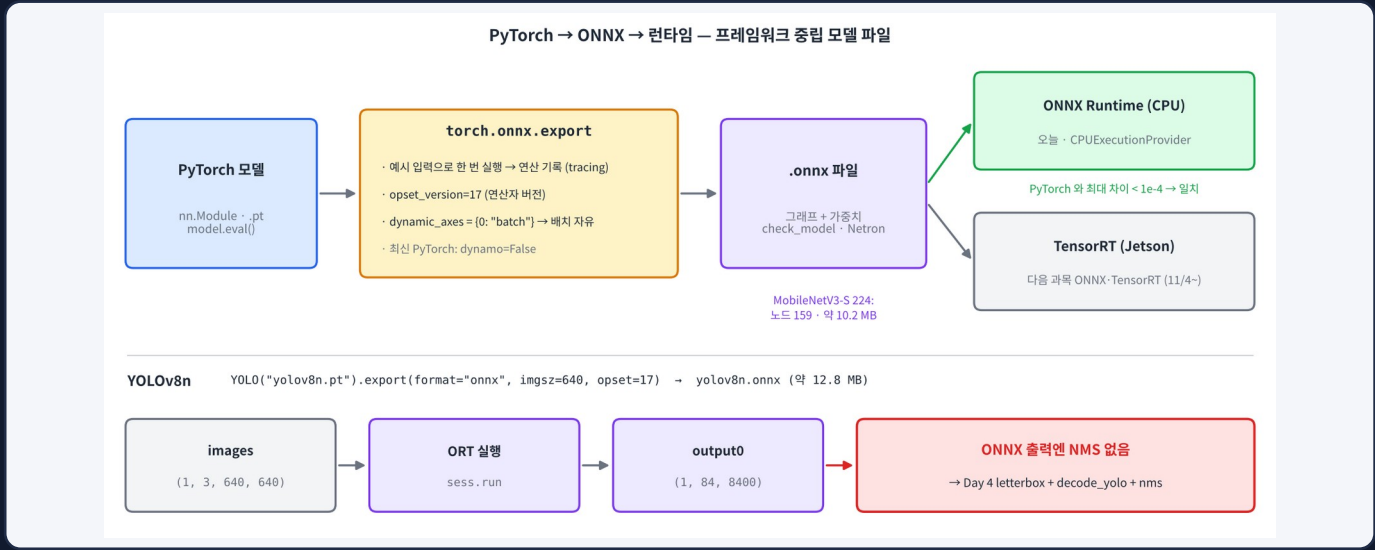
- ONNX = 프레임워크 중립 모델 파일 (그래프 + 가중치)
- export = 예시 입력으로 한 번 실행하며 기록 (tracing)
- dynamic axes = 배치 크기 자유 · opset 17
- ONNX 출력엔 NMS 없음 → Day 4 후처리

노트북 실습

1. 문제 5-1 MobileNetV3 export_onnx
2. 문제 5-2 ONNX Runtime 일치
3. 문제 5-3 YOLOv8n ONNX + decode

▶ 애니메이션 D5-5 ONNX 그래프

핵심 개념



5-1 export

```
torch.onnx.export(model, x, "mnv3.onnx", opset_version=17,
  dynamo=False, input_names=["input"], output_names=["logits"],
  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}})
→ 노드 159 개 · 약 10.2 MB (= 2,542,856 × 4 B)
```

체크포인트

ONNX 그래프에는 NMS 가 없다 — 후처리는 우리 코드 (Day 4) 몫. 오늘 만든 .onnx 를 다음 과목에서 Jetson TensorRT 엔진으로 바꾼다.

6 성능 측정법 — latency · FPS · 스레드

15:00–15:50

학습 목표

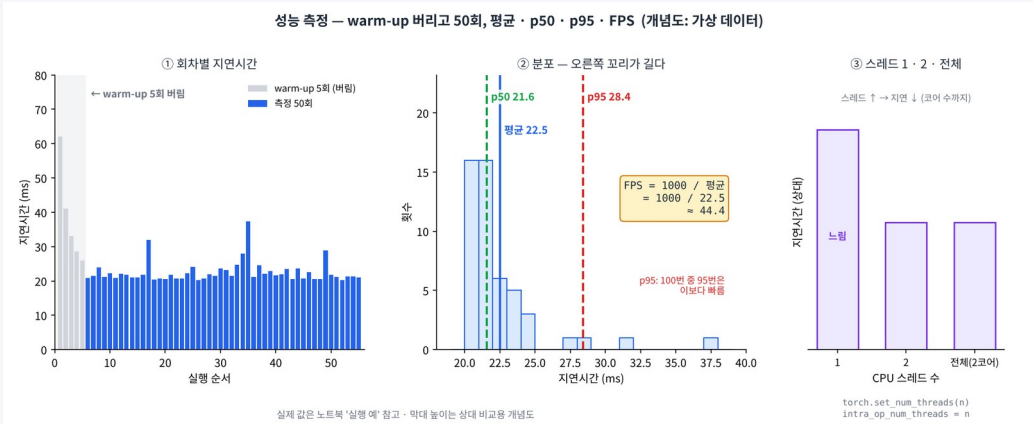
- warm-up 5 회 버림 → 50 회 측정
- 평균 · p50 · p95 (지연시간은 꼬리가 길다)
- FPS = 1000 / 평균 ms · 모델 파일 크기 (MB)
- 스레드 1 · 2 · 전체 → 온디바이스 근사

노트북 실습

1. 문제 6-1 benchmark
2. 문제 6-2 스레드 · 프레임워크 비교
3. 문제 6-3 baseline.csv

▶ 애니메이션 D5-6 latency 분포와 스레드

핵심 개념



6-2 (실행 예)

```
CPU · 배치 1 · 스레드 1 (PyTorch → ORT)
MobileNetV3-S 224  PyTorch 11.0 → ORT 4.6 ms
YOLOv8n 640        PyTorch 219.5 → ORT 159.4 ms
.onnx  mnv3 10.2 MB · yolov8n 12.8 MB
```

6-3

```
baseline.csv  모델 · 프레임워크 · 입력 · 스레드
              mean ms · p95 ms · FPS · 파일 MB
```

체크포인트

평균만 보면 가끔 느린 프레임이 숨는다 → p95 도 본다 . baseline.csv 는 다음 과목 (Jetson · TensorRT) 에서 같은 표로 비교하는 기준이다 .

7 종합 미니 프로젝트 — 위험구역 침입 감지

16:00–16:50

people.mp4 → ONNX Runtime YOLOv8n 사람 탐지 → 발 위치 → ROI 다각형 판정 → 3 프레임 연속이면 경보 → 결과 영상 + CSV

과제	내용	배점
1	구역 판정 in_zone · 발 위치 foot_points	20
2	ONNX 탐지 함수 detect_onnx (letterbox → ORT → decode_yolo → 원래 좌표)	20
3	영상 처리 · 3 프레임 debounce 경보 · 결과 영상 · CSV 로그	20
4	이벤트 요약 + 성능 (ms/ 프레임 · FPS) → baseline 표에 한 줄 추가	20
5	발표 · 해석 (강사)	20

채점 셀 실행 → 자동 채점 80 점 (과제 1~4) + 강사 20 점 (과제 5)

판정 규칙 (768×576 프레임 좌표)

```
GRASS = [(0, 270), (180, 300), (380, 350),
         (520, 470), (560, 575), (0, 575)]
발 위치 = 박스 아래 중앙 ((x1 + x2) / 2, y2)
cv2.pointPolygonTest(poly, pt, False) >= 0 → 안
(100, 400) 안 · (600, 200) 밖 · (300, 340) 안 · (300, 320) 밖
CSV  frame,time_s,n_people,n_inside,alarm
```

실행 예 (CPU) 795 프레임 중 경보 224 프레임 · 이벤트 5 번 · 구역 안 최대 3 명 · 121.5 ms/ 프레임 (8.2 FPS)



▶ 애니메이션 D5-7 ROI 다각형과 침입 판정

8 발표 · 최종 퀴즈 · 마무리

17:00–17:50

학습 목표

- 팀별 결과 영상 · 성능 리포트 발표 (3 분)
- 최종 필기 퀴즈 30 문항 (Day 1~5)
- baseline.csv 제출
- 다음 과목 예고 — ONNX → TensorRT 엔진

노트북 실습

1. 발표 — 결과 영상 · 성능 리포트
2. 최종 퀴즈 30 문항 (Google Forms)
3. baseline.csv 제출

▶ 애니메이션 D5-8 5 일 지도

핵심 개념

발표 3 분

```
1  구역 · 판정 규칙 (발 위치 · 3 프레임 debounce)
2  결과 영상 · 이벤트 (시작 s, 끝 s)
3  성능 ms/프레임 · FPS (CPU) → baseline 표 한 줄
```

제출

```
baseline.csv  모델 · 프레임워크 · 입력 · 스레드
              mean ms · p95 ms · FPS · 파일 MB
```

5 일 지도

```
Day 1  Tensor · autograd · 영상 파이프라인
Day 2  CNN (LeNet · VGG · ResNet)
Day 3  경량 (DS Conv · MobileNet) · 전이학습 · Grad-CAM
Day 4  bbox · IoU · NMS · mAP · YOLO
Day 5  세그 · 포즈 · ONNX · 측정 · 침입 감지
```

체크포인트

다음 과목에서는 오늘 만든 .onnx 를 Jetson 의 TensorRT 엔진으로 바꾸고, baseline.csv 와 같은 표로 속도를 비교한다.

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

박스의 한계	박스 안 상당 부분이 배경 · 경계 침범 = 발 위치 · 쓰러짐은 h/w 로 애매	D5-1
세그멘테이션	semantic vs instance · Encoder-Decoder · TConv 8×8 → 16×16 · mIoU 0.5833	D5-2
포즈	COCO 17 관절 · angle 90° · 몸통 기울기 > 60° → 쓰러짐 (conf ≥ 0.5)	D5-3
태스크 고르기	출력 채널 84 · 116 · 56 · 필요한 출력 → 가장 단순한 태스크 → 속도	D5-4
ONNX	export (dynamic axes · opset 17) · 노드 159 개 · ORT 차이 < 1e-4 · NMS 는 밖에서	D5-5
성능 측정	warm-up 5 · 50 회 · 평균 · p50 · p95 · FPS = 1000 / ms · 스레드 → baseline.csv	D5-6
침입 감지	ONNX YOLOv8n → 발 위치 → pointPolygonTest → 3 프레임 debounce → 영상 + CSV	D5-7
마무리	발표 3 분 · 최종 퀴즈 30 문항 · baseline.csv 제출	D5-8

5 일 돌아보기 — Vision 기본 모델

" 무엇이 " → " 어디에 " → " 어떤 모양 · 어떤 자세 " → " 얼마나 빠르게 " — Tensor 에서 위험구역 침입 감지까지

Day 1

PyTorch · 이미지 · 영상

- Tensor
- autograd
- 영상 파이프라인

Day 2

CNN

- LeNet
- VGG
- ResNet

Day 3

경량 CNN

- DS Conv · MobileNet
- 전이학습
- Grad-CAM

Day 4

객체 탐지

- bbox · IoU · NMS
- mAP
- YOLO

Day 5

세그 · 포즈 · ONNX

- 세그 · 포즈
- ONNX · 측정
- 침입 감지

이 과정에서 만든 것 — 다음 과목에서 그대로 씁니다

`iou_nms_map.py`	영상 파이프라인	`.onnx` 모델	`baseline.csv`	침입 감지 코드
Day 4 · IoU · NMS · mAP	Day 4 · VideoCapture → 오버레이	Day 5 · mnv3 · yolov8n	Day 5 · CPU 성능표	Day 5 · 발 위치 + ROI + debounce

다음 과목으로 — ONNX·TensorRT (11/4~)

오늘 CPU 에서 잰 숫자가 Jetson 에서의 비교 기준이 됩니다 — 같은 표 , 같은 코드

가져가는 것 (Day 5)		다음 과목에서
`baseline.csv (CPU 성능표 )`	→	Jetson TensorRT 엔진과 같은 표로 비교하는 기준
`mnv3.onnx · yolov8n.onnx`	→	ONNX → TensorRT 엔진 변환 (Jetson)
`침입 감지 코드`	→	[프로젝트] 현장 안전 모니터링 (11/25~) 출발 코드 · 입력만 파일 → 카메라
`iou_nms_map.py`	→	후처리 (decode · NMS) 와 mAP 평가에 그대로 사용

ONNX·TensorRT (11/4~, Jetson) → Quantization·Pruning (11/16~) → [프로젝트] 현장 안전 모니터링 (11/25~) · YOLOv8 Edge 모델 (12/11~)

5 일 동안 수고하셨습니다